Solution 1. Below is the integer program requested.

$$
\begin{aligned}
\text{Minimize} \quad & y \\
\text{subject to} \quad & c \in \{0,1\}^{[n]\times V}, y \in \mathbb{Z}_+ \\
& \sum_{i=1}^{n} c_{iv} = 1, && \text{for each } v \in V, \\
& c_{iu} + c_{iv} \leq 1 && \text{for each } \{u,v\} \in E \text{ and each } i \in [n], \\
& y \geq \sum_{i=1}^{n} i c_{iv}, && \text{for each } v \in V.
\end{aligned}
$$

The above formulation uses n^2 variables where, each one is associated with a vertex v and a color c. It also uses 1 variable y, which is used to get the minimum-size coloring of G. If v has color c, the correspondent variable will be equal to 1, otherwise it will be 0.

The first set of constraints $\sum_{i=1}^{n} c_{iv} = 1$ has n equalities and enforces that each vertex has one color. The second set of constraints $c_{iu} + c_{iv} \leq 1$ has $n.m$ inequalities and guarantee that there is no neighboring vertices with the same color. Finally, the last set of constraints y $\geq \sum_{i=1}^{n} i c_{iv}$ has n inequalities and exists to extract the answer. For each inequality in this last set, exactly one variable will be equal to 1 and all the others equal to 0 (one vertex can have only one color). By that, the result of the sum will be i, where i is the index associated with the variable ci, v, meaning that the vertex v has color i. Given that, y will be greater or equal to the maximum index j among all colors utilized. Since we are minimizing y, the best answer will have lower index colors, and thus y will be equal to the minimum number of colors needed to color G.

Solution 2. Supose B_D is not totally unimodular. Then, there are $X \subseteq V$ and $Y \subseteq A$ such that $|X| = |Y|$ and $\det(B_D[X, Y]) \notin \{0, -1, +1\}$. Among all other pairs (X, Y), choose the one that minimizes the size of X and Y. If $|X| = |Y| = 1$, $\det(B_D[X, Y]) \in \{0, -1, +1\}$ because, by definition, all the entries of B_D are an element of $\{0, -1, +1\}$. Hence, $|X| = |Y| \geq 2$. If some column of $B_D = 0$, then $\det(B_D[X, Y]) = 0$. Hence, each column of $B_D[X, Y]$ has at least one entry that is an element of $\{-1, +1\}$. If some column has exactly one 1 or one -1, say at entry $(a, b) \in X \times Y$, then, by Laplace expansion of the determinant, we have $\det(B_D[X, Y]) = \pm\det(B_D[X \setminus \{a\}, Y \setminus \{b\}]) \in \{0, -1, +1\}$. Hence, every column of $B_D[X, Y]$ has exactly one entry equal to -1 and one entry equal to 1. Given that, $(B_D[X, Y])^\intercal \mathbb{1} = 0$, where $\mathbb{1}$ is the vector where all entries are equal to 1. Hence, $(\det(B_D[X, Y]))^\intercal = 0$, and therefore, $\det(B_D[X, Y]) = 0$. Thus, B_D is totally unimodular.

Solution 3. First, it is impossible to have both objects at the same time. If you have a matching that saturates A, you have $|A|$ different edges, one from each element in A to a different element in B. Therefore, if there is a matching that saturates A, the inequality $|S| \leq |N(S)|$ holds for every $S \subseteq A$ because every $S \subseteq A$ has at least $|S|$ elements in B that are neighbors of S. Also, if you have a subset $S \subseteq A$ such that $|S| > |N(S)|$ you can not have a matching that saturates A because there is a vertex v in S such that v is not an end of any edge on a maximum matching of G. This is easy to see because you can not have a matching that saturates S if you have fewer neighbors of S than $|S|$. If you can not have a matching that saturates S, you can not have a matching that saturates A.

The following algorithm solves the problem.

Algorithm 1 Matching-or-Hall-Set $(G, (A, B))$

```
1: (M, K) ← Bip-Matching-and-Vertex-Cover (G, (A, B))
2: if |M| = |A| then
3:     return M
4: end if
5: S ≔ ∅                                  ▷ S will be the subset S ⊆ A such that |S| > |N(S)|
6: for each v ∈ A do
7:     if v ∉ K then                      ▷ this can be done in constant time by using an auxiliary array
8:         S ← S ∪ {v}
9:     end if
10: end for
11: return S
```

The call to Bip-Matching-and-Vertex-Cover algorithm with returns (M, K), where M is a matching of G and K is a vertex cover of G, and $|M| = |K|$. By corollary 21, follow that M is a maximum matching of G and K is a minimum vertex cover of G.

Algorithm 1 runs Bip-Matching-and-Vertex-Cover algorithm, checks if M saturates A, and returns M if the verification is true. Otherwise, we know that no matching of G saturates A, and we need to return a subset $S \subseteq A$ such that $|S| > |N(S)|$. In Algorithm 1, the subset S is define as $S \coloneqq \{a \in A : a \notin K\}$.

First, S is a subset of A. Second, $N(S)$ is the set of all elements $b \in B$ such that $b \in K$. If $b \in B$ and $b \in K$, then b has at least one neighbor that is not in K, otherwise K would not be a minimum vertex cover because you could remove b from K, and it would still be a vertex cover, which is a contradiction. Therefore, b has a neighbor in S and, by definition, is in $N(S)$.

Finally, we will prove that $|S| > |N(S)|$. An important property of this setup is that $|M| = |K|$ and $|M| < |A|$, thus $|K| < |A|$. Set $K_A \coloneqq \{a \in A : a \in K\}$ and $K_B \coloneqq \{b \in B : b \in K\}$. Given that K_A and K_B are disjoint, we have $|K_A| + |K_B| < |A|$. Working on that, trying to have an inequality where one side is a subset of A, we have $|K_B| < |A| - |K_A|$. Because $K_A \subseteq A$, then $|A| - |K_A| = |A \setminus K_A|$. Knowing that $K_B \supseteq N(S)$ and $A \setminus K_A = S$, it is proved that $|N(S)| < |S|$.

Regarding the time analysis, lines 2, 3, and 4 can be done in constant time if you store the size of the set (in the worst case, the IF verification is linear on the number of elements of the sets). The FOR loop on line 5 runs $|A|$ times, and the IF on line 6 can be done in constant time by using an auxiliary boolean array that stores whether a vertex v is in K or not (it costs linear time on $|V(G)|$ to build the array plus linear time on $|K|$ to fill the array). Concluding, besides the running time of Bip-Matching-and-Vertex-Cover, Algorithm 1 runs in time $O(n + m)$ and does not run any graph traversal algorithm.

Solution 4. Set C to be the set of all convex combinations of points of X. To prove that $\text{conv}(X) = C$, I will prove that $\text{conv}(X) \subseteq C$ and that $C \subseteq \text{conv}(X)$. Also, consider an injective function $f : X \mapsto [n]$, where n is the size of X.

To prove that $\text{conv}(X) \subseteq C$, I will prove that $X \subseteq C$ and it is convex. Given that $\text{conv}(X)$ is the intersection of all convex subsets of $\mathbb{R}^V$ that contain X, if $X \subseteq C$ and C is convex, then $\text{conv}(X) \subseteq C$.

First, a point y is convex combination of points of X if there is $\lambda \in \mathbb{R}^{\text{I}}_+$ such that $\mathbb{1}\lambda = 1$ and $y = \sum_{i=1}^{n} \lambda_i x_i$. Given this definition is easy to see that every $x \in X$ is a convex combination of points of X (choosing $\lambda_i = 1$, the sum will be equal to x_i).

Finally, let $y_1 \coloneqq \sum_{i=1}^{n} \alpha_i x_i$ and $y_2 \coloneqq \sum_{i=1}^{n} \beta_i x_i$ be arbitrary convex combinations of points of X and let $\lambda \in [0, 1]$. We will prove that $(1-\lambda)y_1 + \lambda y_2$ is in C. Writing the expression for y_1 and y_2 we have $(1-\lambda)y_1 + \lambda y_2 = (\sum_{i\in \text{I}}(1-\lambda)\alpha_i x_i) + (\sum_{i\in \text{I}} \lambda\beta_i x_i)$. Merging the two sums we have $(1-\lambda)y_1 + \lambda y_2 = \sum_{i\in \text{I}}(1-\lambda)\alpha_i x_i + \lambda\beta_i x_i$. Finally we have $(1-\lambda)y_1 + \lambda y_2 = \sum_{i\in \text{I}}((1-\lambda)\alpha_i + \lambda\beta_i)x_i$.

If $\sum_{i\in \text{I}}((1-\lambda)\alpha_i + \lambda\beta_i) = 1$, then $(1-\lambda)y_1 + \lambda y_2$ is a convex combination of points of X. We have that $\sum_{i\in \text{I}}((1-\lambda)\alpha_i + \lambda\beta_i) = \sum_{i\in \text{I}}(\alpha_i - \lambda\alpha_i + \lambda\beta_i)$. Calculating we have $\sum_{i\in \text{I}}((1-\lambda)\alpha_i + \lambda\beta_i) = \sum_{i\in \text{I}} \alpha_i - \sum_{i\in \text{I}} \lambda\alpha_i + \sum_{i\in \text{I}} \lambda\beta_i$. Substituting values we have $\sum_{i\in \text{I}}((1-\lambda)\alpha_i + \lambda\beta_i) = 1 - \lambda + \lambda = 1$. Thus, we proved that $\text{conv}(X) \subseteq C$.

To prove that $C \subseteq \text{conv}(X)$, I will prove that an arbitrary convex combination of points of X is a point that is in every convex subset of $\mathbb{R}^V$ that contain X, and therefore is in $\text{conv}(X)$.

By defintion of convex, $y_1 \coloneqq \lambda_1 x_1 + (1-\lambda_1)x_2$, where $x_1, x_2 \in X$ and $\lambda_1 \in [0, 1]$, is in $\text{conv}(X)$. Also, $y_2 \coloneqq \lambda_2 y_1 + (1-\lambda_1)x_3$, where $x_3 \in X$ and $\lambda_2 \in [0, 1]$, is in $\text{conv}(X)$ because y_1 and x_3 are points in $\text{conv}(X)$. By adding all the n vertices using the same logic, we have that

$$y \coloneqq (\prod_{k=1}^{n-1} \lambda_k)x_1 + (\sum_{i=2, i\neq j}^{n-1} (\prod_{k=i}^{n-1} \lambda_k)(1-\lambda_{i-1})x_i) + (1-\lambda_{n-1})x_n$$

is in $\text{conv}(X)$.

First,

$$(\prod_{k=1}^{n-1} \lambda_k) + (\sum_{i=2, i\neq j}^{n-1} (\prod_{k=i}^{n-1} \lambda_k)(1-\lambda_{i-1})) + (1-\lambda_{n-1}) = 1$$

holds. You can verify that by simply putting in evidence the two terms with more lambdas and repeating the process. Every time you do that you will have $\lambda_i + 1 - \lambda_i$ and a smaller expression.

Finally, an arbitrary convex combination $c \coloneqq \sum_{i=1}^{n} \alpha_i x_i$ can be written as

$$(\prod_{k=1}^{n-1} \lambda_k)x_1 + (\sum_{i=2, i\neq j}^{n-1} (\prod_{k=i}^{n-1} \lambda_k)(1-\lambda_{i-1})x_i) + (1-\lambda_{n-1})x_n$$

.

I will show that this holds. Choose a function f such that $\alpha_i > \alpha_j$ if $i > j$. In this generic setting $\alpha_n > 0$. Let $(1-\lambda_{n-1}) \coloneqq \alpha_i$. You should also multiply every other x_i, where $i \in [n-1]$ by λ_{n-1}. At this poin the sum between α_n and the coeficient of an arbitrary x_i is 1. You will repeat this procedure which you pick each x_i in order from $i = n-1$ to $i = 2$, multiply it by $(1-\lambda_{i-1})$, where $(1-\lambda_{i-1})$ is the number which $(1-\lambda_{i-1})\prod_{k=i}^{n-1} \lambda_k = \alpha_i$. For x_{n-1} it is clear that this is possible because $(1-\lambda_{n-2}) \in [0, 1]$, so the coeficient of x_{n-1} can be set to the correct value α_{n_1} because $\alpha_{n_1} \in [0, 1-\alpha_n]$. This holds for every iteration because $\alpha = 1 - a$ implies that $\lambda\alpha = 1 - a - (1-\lambda)\alpha$ for $a, \alpha, \lambda \in [0, 1]$. Given that, in every iteration an arbitrary coeficient will be equal to 1 minus the sum of all seted coeficients. In addition, you will always be able to set the correct value for a coeficient because it will be always equal or smaller of its current value, otherwise the previous statement would not hold. Also, the sum of coeficients will be 1 at the end because it match our expression. The last coeficient, because of that, will have the right value.

Given the discussion, we can conclude that $\text{conv}(X) = C$.

Solution 5. Following the hint, I will prove that $b(b(\mathcal{H}))^{\uparrow} = \mathcal{H}^{\uparrow}$. For that, I will show that $b(b(\mathcal{H}))^{\uparrow} \subseteq \mathcal{H}^{\uparrow}$, and then that $\mathcal{H}^{\uparrow} \subseteq b(b(\mathcal{H}))^{\uparrow}$. Note that, by definition, the hyperedges of $b(b(\mathcal{H}))^{\uparrow}$ are all subsets of vertices of $\mathcal{H}$ that have at least one vertex of each minimal vertex cover of $\mathcal{H}$. On the other hand, the hyperedges of $\mathcal{H}^{\uparrow}$ are all subsets of vertices of $\mathcal{H}$ that contains at least one hyperedge of $\mathcal{H}$.

To prove that $b(b(\mathcal{H}))^{\uparrow} \subseteq \mathcal{H}^{\uparrow}$, I will prove that a non-hyperedge of $\mathcal{H}^{\uparrow}$ is non-hyperedge of $b(b(\mathcal{H}))^{\uparrow}$. An arbitrary set of vertices s is not a hyperedge of $\mathcal{H}^{\uparrow}$ if, for all $e \in \mathcal{E}$, holds $e \nsubseteq s$. If exists a minimal vertex cover cov of $\mathcal{H}$ such that $s \cap cov = \varnothing$, then s is a non-hyperedge of $b(b(\mathcal{H}))^{\uparrow}$ because, by definition, a hyperedge of $b(b(\mathcal{H}))^{\uparrow}$ need to have at least one element of each minimal vertex cover. Let cov be a set of vertices that is built by including one element of each $e \in \mathcal{E}$ that is not in s, for sure cov is a vertex cover. Then, get $cov_{\min}$, a minimal set of $v \in V$ such that $cov_{\min}$ is a vertex cover and every element in $cov_{\min}$ is in cov. This can be done by removing each element $x \in cov$ such that $cov \setminus \{x\}$ is a vertex cover of $\mathcal{H}$. Given that $cov_{\min} \cap s = \varnothing$, then $b(b(\mathcal{H}))^{\uparrow} \subseteq \mathcal{H}^{\uparrow}$.

To prove that $\mathcal{H}^{\uparrow} \subseteq b(b(\mathcal{H}))^{\uparrow}$, I will prove that if a set of vertices is a hyperedge of $\mathcal{H}^{\uparrow}$, then it is also a hyperedge of $b(b(\mathcal{H}))^{\uparrow}$. By definition, all minimal vertex covers of $\mathcal{H}$ contains at least one element of each hyperedge of $\mathcal{H}$. More than that, for each hyperedge $e \in \mathcal{E}$ there is a minimal vertex cover $cov_{\min}$ such that $e \cap cov_{\min} \neq \varnothing$. This is true because you can made this minimal vertex cover by starting with $\{v\}$, where v is an arbitrary vertex in e, and continue adding vertices of hyperedges that are not covered by the current elements. As you are adding only vertices which cover new hyperedges, you will stop adding when the set becomes a vertex cover. Because you can not remove any element, the set is a minimal vertex cover of $\mathcal{H}$. Given that, every $e \in \mathcal{E}$ is a hyperedge in $b(b(\mathcal{H}))^{\uparrow}$. By definition, every hyperedge of $\mathcal{H}^{\uparrow}$ is also a hyperedge of $b(b(\mathcal{H}))^{\uparrow}$.

By the discussion above, $b(b(\mathcal{H}))^{\uparrow} = \mathcal{H}^{\uparrow}$. Applying the definition we can state that $(b(b(\mathcal{H}))^{\uparrow})^{\min} = (\mathcal{H}^{\uparrow})^{\min}$. By exercise 11 of assignment 0 we have $(b(b(\mathcal{H})))^{\min} = \mathcal{H}^{\min}$. As the hypergraph $b(b(\mathcal{H}))$ has a set of hyperedges only with minimal hyperedges, we have $b(b(\mathcal{H})) = \mathcal{H}^{\min}$.